# Week 6, Tuesday: numpy arrays, reductions, grids and the dot product



These notes cover week 6's first lecture, numpy, for revision. Read them beside its slides: every part names the pages it covers. They add the explanation behind each slide, the runs written out line by line, the mistakes the code invites with the message Python prints, the polls with their answers, and a few questions to check yourself, answered at the end. Common questions are answered where they come up. Others have written answers on the Q&A page, and the Function reference lists what each tool takes and gives back; both are on Brightspace, under Feedback. If you already program, the boxes headed **Good practice** are for you: the habits experienced programmers keep, next to the code they are about. The notes replace neither the lecture nor the practical: the sheet is where you practice.

## Where we started

*Slides, pages 3 and 5.*

Week 5 left the café unkillable: bad input, a missing file and zero guests no longer bring the register down. One trap was left, ordering a product that has sold out: the week-5 notes show it, and this week's café files close it. So the owner raised the bar. She wants numbers: the day's revenue, the average basket, the biggest order and which order it was, how spread out the orders are, the items sold in the week, the busiest hour and the biggest day. Ten questions, and with week 4's tools, ten hand-written loops. numpy throws the loops away: it answers each question in one line, on data of any size. It also settles a debt from week 4, the bill computed with a loop that was meant to take one line one day. Section 5 pays it.

Week 5's follow-up is on the Q&A page, under Week 5.

## 1 · Arrays and reductions

*Slides, pages 6 to 12.*

### Why Python is slow on millions of numbers

*Page 6.* Python has served us well for five weeks: short programs, quick to write and easy to read; mixed data handled for free, since the café's order list holds tuples of strings and ints side by side; memory managed for us by week 1's garbage collector; and routines ready to call, `len`, `sum`, `sorted`. That flexibility has a price. Because a list can hold anything, Python checks the type of every element before every operation, and every value is a full object with its own overhead, week 5's boxes. For a register serving one customer at a time this costs nothing you would notice. For millions of numbers, a chain's year of transactions or a market's tick data, it makes Python slower than languages built for number-crunching.

## numpy's deal

*Page 7.* numpy keeps much of Python's ease and wins back the speed with one data structure, the **array**, handled by fast, pre-compiled routines. The array's deal:

- **typed**: every element has the same type, so every element takes the same number of bytes;
- **contiguous**: all the data sits in one continuous block of memory, so finding element `i` is arithmetic, not a search;
- one type everywhere, so a routine runs over **all the elements without checking each one**, often several at once.

"Several at once" happens inside one processor core, which works on a few numbers per step: no extra cores needed. The checklist on page 41 puts the speed in four words: typed, contiguous, pre-compiled, whole-object. Each one comes back below.

## The first array

*Page 8.* Type this into the shell:

```
>>> import numpy as np
>>> totals = np.array([12.10, 6.40, 15.80, 3.20])
>>> print(totals)
[12.1  6.4 15.8  3.2]
>>> print(type(totals))
<class 'numpy.ndarray'>
```

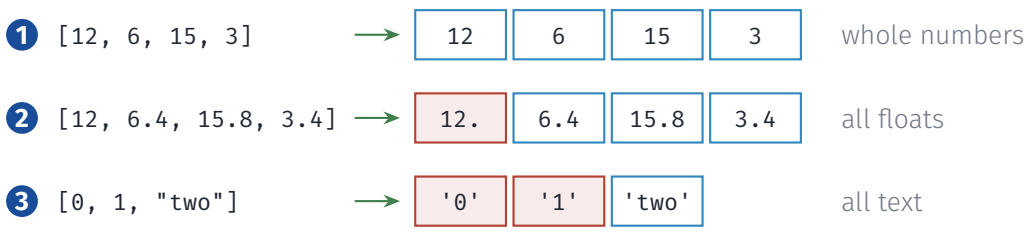
`import numpy as np` gives the module a short name, and everyone uses that one. `np.array(...)` builds an array from a list. The print shows numpy's own layout: spaces and no commas, the trailing zero of `12.10` dropped, and every value padded to the same width. The type is `numpy.ndarray`, an object with methods of its own, the pre-compiled ones.

## One type for every element

*Page 8.* What if the list mixes types? Run these three:

```
>>> print(np.array([12, 6, 15, 3]))
[12  6 15  3]
>>> mixed = np.array([12, 6.4, 15.8, 3.4])
>>> print(mixed)
[12.   6.4 15.8  3.4]
>>> print(type(mixed[0]))
<class 'numpy.float64'>
>>> print(np.array([0, 1, "two"]))
['0' '1' 'two']
```

❶ Whole numbers only, and the array holds whole numbers: no dots. ❷ One float among them, and every element becomes a float: the `12` comes back as `12.`, and each element is a `numpy.float64`, a float stored in 64 bits, 8 bytes, the same for every element. ❸ One string, and every element becomes text: `'0'` and `'1'` in quotes. An array holds one type, so numpy picks the type that can hold every value and converts the rest.

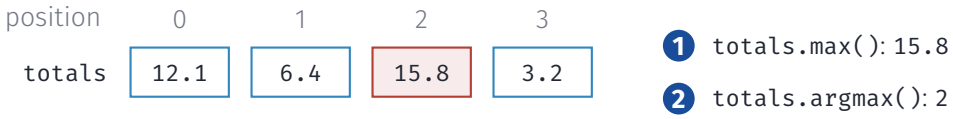


## The ten questions, answered

*Page 9.* Every question on the owner's list is a **reduction**: many numbers in, one number out. Arrays come with reductions built in:

```
>>> totals = np.array([12.10, 6.40, 15.80, 3.20])
>>> print(totals.min(), totals.max(), totals.sum(), totals.mean())
3.2 15.8 37.5 9.375
>>> print(totals.std())
4.8909993866284625
>>> print(totals.argmax())
2
```

Four lines answer half the list, and `.std()` is the standard deviation, how spread out the orders are. The last line deserves a second look. (1) `.max()` reads the four values, `12.1`, `6.4`, `15.8` and `3.2`, and returns the biggest, `15.8`. (2) `.argmax()` returns where that value sits, position `2`, counting from 0 as every index in Python does. One letter apart, two different questions: how big, and which one.




## `day_stats()`: into the register

*Page 10.* The register now remembers every order total of the day, in a list, and option `[6] Day stats` turns that list into four numbers:

```
def day_stats(order_totals):
    """Some quick numbers about a day's orders, with numpy."""
    values = np.array(order_totals)
    return {"orders": len(values), "revenue": round(float(values.sum()), 2),
            "biggest": round(float(values.max()), 2), "average": round(float(values.mean()), 2)}
```

In `main()`, every sale appends its total, `order_totals.append(paid)`, beside the lines that add to the revenue and save the receipt. A week-4 list feeds a numpy array, which feeds a week-4 dictionary. Each value passes through `float(...)` before `round(..., 2)`: numpy's sum is a numpy number, and a dictionary prints a numpy number with its type showing. Unconverted, the revenue would print as `np.float64(13.9)`; converted, it prints as `13.9`.

Try it: start `python3 cafe_w6a.py` from your café folder, type `Ada` as the cashier, then take two orders. Order one: `1`, Enter past the customer's name, `latte`, `2`, `done`, Enter past the split. That is 6.40. Order two: `1`, Enter, `espresso`, `3`, `done`, Enter: 7.50. Then choose `6`:

```
  Today so far: {'orders': 2, 'revenue': 13.9, 'biggest': 7.5, 'average': 6.95}
```

Choose `0` to close. Option 6 reads only `order_totals`, so revenue carried over from an earlier day does not change these four numbers.

**Good practice.** Check for an empty array before you reduce it. With no order yet, `np.array([]).mean()` returns `nan` and warns `Mean of empty slice`, and `np.array([]).max()` stops:

```
ValueError: zero-size array to reduction operation maximum which has no identity
```

The café tests `if order_totals == []:` first and prints `No orders yet today.` instead of reducing nothing.

## The numpy way and the Python way

*Page 11.* Both programs compute a mean of ten million numbers:

```
# ok.py: Python's sum() and len() work on arrays...
x = np.array(list(range(10000000)))
print(sum(x) / len(x))
```

```
# better.py: ...but the array already knows how
x = np.array(list(range(10000000)))
print(x.mean())
```

Both print `4999999.5`, and `x.mean()` gets there far faster. `sum()` is Python's own, so `sum(x)` fetches and checks ten million numbers one at a time, while `x.mean()` runs numpy's compiled routine over the one block. The rule of thumb: if numpy has a method for it, use numpy's method.

**Poll.** What does `print(np.array([10, 20, 30]).mean())` show: `20.0`, `20`, `60.0`, or an error? It shows `20.0`. A mean is a reduction, three numbers in and one out, and it comes back as a float, even when the numbers going in are whole and the average is too.

## Predict, then run

*Page 12.* The café's menu as an array, and five lines to predict before running them:

```
prices = np.array([2.50, 3.20, 3.00, 2.80, 2.40, 2.20])
print(prices.max())          # 1
print(prices.argmax())       # 2
print(prices.sum())          # 3
print(prices.mean())         # 4
print(len(prices))           # 5
```

**Poll.** Predict `prices.argmax()`: `1`, `3.2`, `2`, or an error? It is `1`: the position of the dearest item, the latte at 3.20. `3.2` is what `.max()` gives, and `2` counts the positions from one.

The five lines print `3.2`, `1`, `16.1`, `2.6833333333333336` and `6`. The mean carries a long decimal tail, and a price wants two decimals: numpy computes, and formatting stays our job, with week 2's `:.2f`.

**Common mistakes.** Taking `.argmax()` for the value: it is a position, and `totals[2]` is the value it points to. Expecting commas in an array's print: `[12.1  6.4 15.8  3.2]` is how numpy writes it, and an answer written with commas is a different output.

**Check yourself.**

- **1.1** In `day_stats`, each value goes through `float(...)` before `round(...)`. What would the dictionary show for the revenue without it, and why?
- **1.2** On ten million numbers, which is faster, `sum(x) / len(x)` or `x.mean()`, and why, when both give the same answer?
- **1.3** After `tips = np.array([3, 2.5])`, what do `print(tips)` and `print(type(tips[0]))` show?

**See also.** Sheet 6, exercise 3. The Function reference, Week 6: Building arrays and Reductions.

## 2 · Vector arithmetic and building arrays

*Slides, pages 13 to 18.*

### Arithmetic on the whole array

*Page 13.* Arithmetic on an array applies to every element, with no loop in sight:

```
>>> x = np.array([1.0, 2.0, 3.0, 4.0])
>>> print(x + 1)
[2. 3. 4. 5.]
>>> print(x * 3)
[ 3.  6.  9. 12.]
>>> print(x * x)
[ 1.  4.  9. 16.]
```

`x + 1` adds one to each element and `x * 3` triples each. `x * x` pairs two arrays position by position: element 0 with element 0, element 1 with element 1. In café terms, putting every price up by 10 cents is `prices + 0.10`, and a happy hour at 10% off is `prices * 0.9`: the whole menu repriced in one line.

Arithmetic needs numbers. Multiply the text array from section 1 by itself:

```
>>> words = np.array([0, 1, "two"])
>>> print(words * words)
```

Nothing prints: the line stops with numpy's `_UFuncNoLoopError`, a `UFuncTypeError` and so a kind of week 5's `TypeError`, which `except TypeError:` catches. Its message ends `did not contain a loop with signature matching types (dtype('<U21'), dtype('<U21')) -> None`: `<U21` is numpy's name for text of up to 21 characters, and `multiply` has no routine for two of them. A Python loop over the list `[0, 1, "two"]` would multiply 0 by 0 and 1 by 1, then crash at `"two"`. numpy checks the types for the whole operation when the line runs, before it computes a single element.

### The no-loops rule

*Page 14.* The same discipline on ten million floats:

```
# bad.py: a Python loop over a numpy array, the worst of both worlds
x = np.array([float(n) for n in range(10000000)])
for i in range(len(x)):
    x[i] /= 10
```

```
# good.py: let the array do it
x = np.array([float(n) for n in range(10000000)])
x /= 10
```

`bad.py` pays the Python-object toll ten million times; `good.py` runs once, compiled. The building block `[float(n) for n in range(...)]` is week 4's list comprehension, making the test data. When you reach for a loop over an array, look for the whole-array line first.

> **Good practice.** Build floats when you plan to divide in place. `x /= 10` writes the result back into `x`, and an array of whole numbers cannot hold a tenth: `np.arange(4)` followed by `/= 10` stops with an error that ends `Cannot cast ufunc 'divide' output from dtype('float64') to dtype('int64') with casting rule 'same_kind'`. `good.py` builds its numbers with `float(n)` for that reason. On whole numbers, write `x = x / 10`, which makes a new array of floats.

## Element-wise, by hand

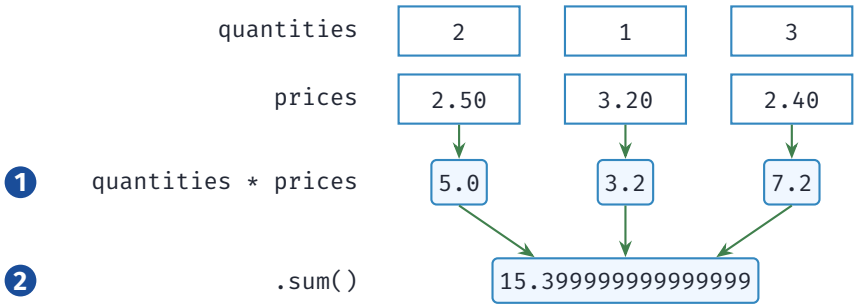
*Page 15.* An order of 2 espressos, 1 latte and 3 muffins: what is the bill? Two arrays, one element-wise multiplication, one reduction. Trace every cell before running it:

```
quantities = np.array([2, 1, 3])
prices     = np.array([2.50, 3.20, 2.40])
line_totals = quantities * prices
print(line_totals, line_totals.sum())
```

| position | quantities | prices | quantities * prices |
|---|---|---|---|
| 0 | 2 | 2.50 | ? |
| 1 | 1 | 3.20 | ? |
| 2 | 3 | 2.40 | ? |
| | | .sum() | ? |

**Poll.** What is `(quantities * prices).sum()`? Write your number down before reading on.

❶ `*` multiplies the two arrays seat by seat: 2 × 2.50 is 5.0, 1 × 3.20 is 3.2 and 3 × 2.40 is 7.2. ❷ `.sum()` reduces the three to one, the bill: 15.4, which Python prints as `15.399999999999999`. The tail is week 1's float imprecision: 3.20 and 2.40 have no exact binary form, and their tiny errors add up.




The run shows both:

```
[5.  3.2 7.2] 15.399999999999999
```

Now give the order a fourth price that has no quantity:

```
>>> quantities = np.array([2, 1, 3])
>>> prices = np.array([2.50, 3.20, 2.40, 2.50])
>>> print(quantities * prices)
ValueError: operands could not be broadcast together with shapes (3,) (4,)
```

Element by element needs the same shape. A week-4 loop over `range(len(quantities))` would make three line totals and never look at the fourth price, without a word. numpy checks the shapes first, so the mismatch cannot slip through.

## Building arrays: the basics

*Page 16.* You will not always start from a list. numpy builds arrays for you:

```
>>> empty_tray = np.zeros(5)
>>> print(empty_tray)
[0. 0. 0. 0. 0.]
>>> lattes = np.ones(5)
>>> lattes *= 3.20
>>> print(lattes)
[3.2 3.2 3.2 3.2 3.2]
>>> nines = np.repeat(9, 5)
>>> print(nines)
[9 9 9 9 9]
```

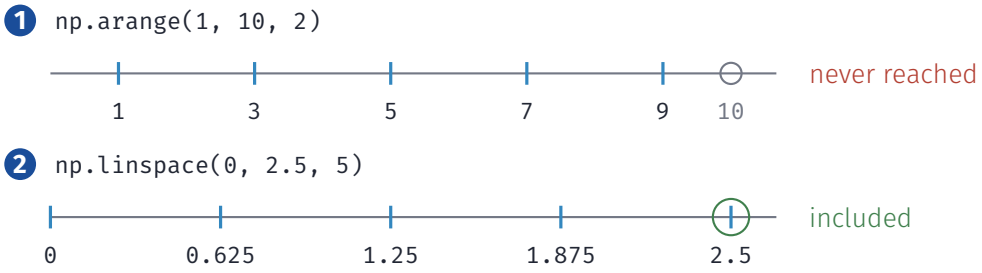
`np.zeros(5)` is a tray with five empty slots, and `np.ones(5)` scaled by 3.20 is five lattes' worth of price. `np.repeat(9, 5)` repeats one value. Look at the dots: `zeros` and `ones` give floats by default, `[0. 0. 0. 0. 0.]`, while `repeat(9, 5)` gives whole numbers, `[9 9 9 9 9]`. An array takes the type of what you feed it.

## Building arrays: sequences

*Page 17.* Two builders make sequences, and they end in opposite ways:

```
>>> seq = np.arange(1, 10, 2)
>>> print(seq)
[1 3 5 7 9]
>>> grid = np.linspace(0, 2.5, 5)
>>> print(grid)
[0.    0.625 1.25  1.875 2.5  ]
```

❶ `np.arange(start, stop, step)` is week 3's `range()` as an array, and like `range` it never reaches its stop: `np.arange(1, 10, 2)` steps by 2 from 1 and ends at 9. ❷ `np.linspace(first, last, n)` takes no step: you say how many values you want, and numpy spaces them evenly with both ends included, so the 2.5 is in.




## Random arrays

*Page 18.* numpy can also draw whole arrays from probability distributions, in one call:

```
gaps = np.random.uniform(1, 10, 4)     # 4 floats in [1, 10), all equally likely
demand = np.random.normal(42, 8, 3)    # a bell curve: mean 42, standard deviation 8
print(gaps)
print(demand)
```

Run it twice and the numbers change: that is what random means. Nor will the three demands average exactly 42: the 42 and the 8 describe the curve the numbers are drawn from, and a small sample wanders around them. This is a toy. Real forecasting needs real statistics, from your other

modules. Simulating "what if Saturday is 20% busier?" on plausible fake numbers is still a useful trick, and the project's market data was generated this way.

**Common mistakes.** Expecting `np.arange` to reach its stop: like `range`, it stops before. Reading `[9 9 9 9 9]` as floats: no dots, whole numbers. Writing a loop over an array where one line does the work. And multiplying arrays of different lengths, which numpy refuses.

**Check yourself.**

- **2.1** What do these two lines print?

```
print(np.arange(10, 0, -3))
print(np.linspace(1, 2, 3))
```

- **2.2** With `menu = np.array([2.40, 3.00])`, write one line that takes 20% off every price, and say what it prints.

**See also.** Sheet 6, exercises 1 and 2. The Function reference, Week 6: Building arrays and Arithmetic on the whole array.

## 3 · Two dimensions

*Slides, pages 20 to 27.*

### One rule: the grid must be regular

*Page 20.* Like week 4's nested lists, numpy builds arrays with more than one dimension, with two constraints: one type everywhere, as before, and a **homogeneous shape**, every row the same length and every column the same length.

```
>>> x = np.array([[1, 2, 3], [4, 5, 6]])
>>> print(x)
[[1 2 3]
 [4 5 6]]
>>> print(x[0], type(x[0]))
[1 2 3] <class 'numpy.ndarray'>
```

Two rows of three, and each row is itself an array. A week-4 list of lists takes ragged rows without complaint; numpy refuses them:

```
>>> print([[1, 2, 3], [4, 5]])
[[1, 2, 3], [4, 5]]
>>> x = np.array([[1, 2, 3], [4, 5]])
ValueError: setting an array element with a sequence. The requested array has an inhomogeneous shape after 1
    ↪ dimensions. The detected shape was (2,) + inhomogeneous part.
```

Read that `ValueError` once now: it comes back whenever rows of different lengths reach `np.array`.

## Building in 2-D

*Page 21.* `zeros`, `ones` and the other builders take a shape as a tuple, week 4's tuples doing structural work. A shape lists the rows first, then the columns: `(2, 5)` is two rows of five.

```
>>> x = np.ones((2, 5))
>>> print(x)
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
>>> print(x + 10)
[[11. 11. 11. 11. 11.]
 [11. 11. 11. 11. 11.]]
```

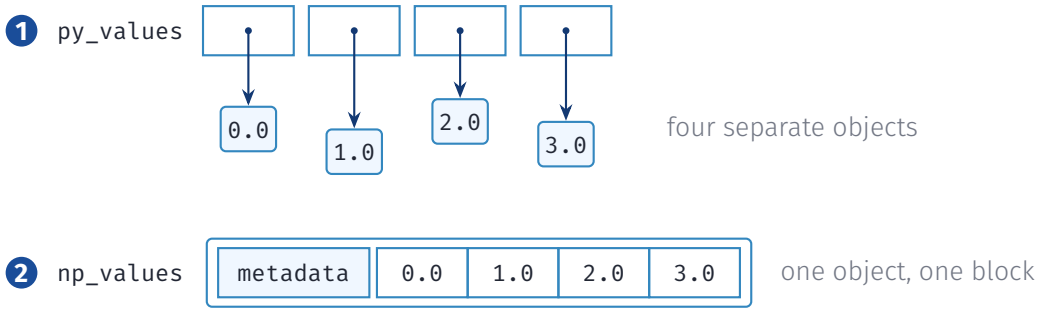
Arithmetic stays whole-object in two dimensions, and the columns line up in the print. numpy does not stop at two: `np.ones((2, 3, 4))` is three-dimensional, two blocks of three rows by four columns, which you can read as weeks by days by hours.

## Under the floorboards

*Page 22.* Week 5's boxes explain the speed. Picture memory as a lecture theater: every seat has a number, its address, and a value sits in a seat.

```
>>> py_values = [float(x) for x in range(4)]
>>> np_values = np.arange(4, dtype=float)
>>> print(py_values)
[0.0, 1.0, 2.0, 3.0]
>>> print(np_values)
[0. 1. 2. 3.]
```

`dtype=float` names the element type. (1) A list books its values wherever seats are free, and its own four seats hold only arrows to them: four separate objects. (2) An array books one run of consecutive seats for its four values, side by side, with the seat in front holding the **metadata**: the element type, how many bytes each element takes and the length of each dimension. The same block can then be read in more than one way, which the rest of this section uses.



### `.shape`, `.size` and `reshape`

*Page 23.* Three questions to ask an array about its dimensions:

```
>>> x = np.arange(10)
>>> print(x, x.shape, x.size, len(x))
[0 1 2 3 4 5 6 7 8 9] (10,) 10 10
>>> x = x.reshape(2, 5)
>>> print(x, x.shape, x.size, len(x))
[[0 1 2 3 4]
 [5 6 7 8 9]] (2, 5) 10 2
```

`.shape` is the length of each dimension, as a tuple: `(10,)` is one dimension, ten long, written as week 4's one-element tuple, comma and all. `.size` is the number of elements, still 10 after the reshape. `len()` gives only the first dimension, so it went from 10 to 2: one more reason to prefer numpy's own `.shape` and `.size`. And `reshape` copied nothing: it handed back a new way to read the same block, new metadata over the old data.

> **Good practice.** Print an array's `.shape` before you compute with it. Most numpy bugs are shape bugs: a grid read the wrong way round, a column where you meant a row, a file that loaded as one long row. `print(grid.shape)` costs one line and tells you which question your next line will answer.

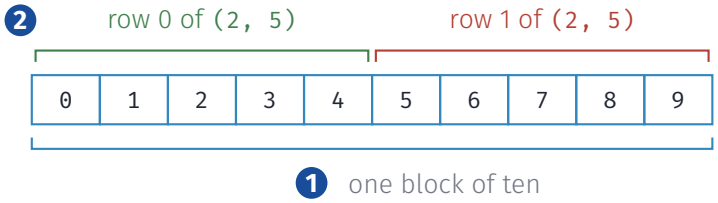
### Three roads to the same 2 × 5

*Page 24.* Three constructions of one grid:

```
x = np.array([[0, 1, 2, 3, 4], [5, 6, 7, 8, 9]])   # 1. born 2-D, from nested lists
y = np.arange(10)                                    # 2. born flat...
y = y.reshape(2, 5)                                  #    ...reshaped by the method
z = np.reshape(np.arange(10), (2, 5))                # 3. reshaped by the function
```

All three print the same grid. Roads 2 and 3 are one operation in two spellings: the method `y.reshape(2, 5)`, and the function `np.reshape(array, shape)`, handy in the middle of an expression.

❶ The block holds ten values, 0 to 9, in a row. ❷ Read as shape `(2, 5)`, its first five values are row 0 and its last five are row 1: the same memory, read as a grid, which is why a reshape costs nothing.



## A reshape is a view

*Page 25.* Week 5's aliasing bug had two names for one list. Here it is again: two shapes, one block. Write down both prints before you run it:

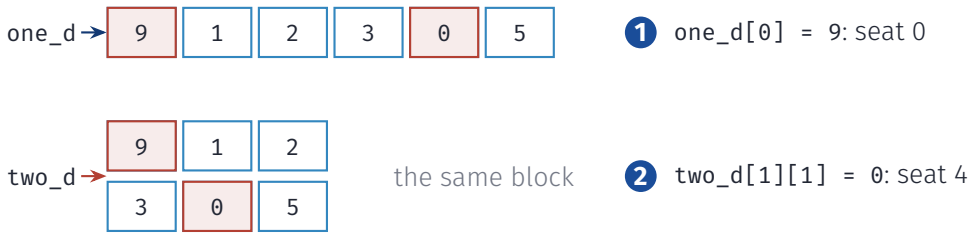
```
one_d = np.arange(6)                # [0 1 2 3 4 5]
two_d = np.reshape(one_d, (2, 3))   # 2 rows of 3, a view of the same data
one_d[0] = 9                        # write through the flat name...
two_d[1][1] = 0                     # ...and through the grid name
print(one_d)
print(two_d)
```

**Poll.** After the two writes, what does `print(one_d)` show? Commit to an answer before reading on.

It shows `[9 1 2 3 0 5]`, and `two_d` prints the same data as a grid:

```
[9 1 2 3 0 5]
[[9 1 2]
 [3 0 5]]
```

1 `one_d[0] = 9` writes the block's seat 0, and `two_d` sees it at row 0, column 0. 2 `two_d[1][1] = 0` writes row 1, column 1 of a grid three wide, which is seat $1 \times 3 + 1$, seat 4, and `one_d` sees it there. Both writes reach both names: one block, read two ways.



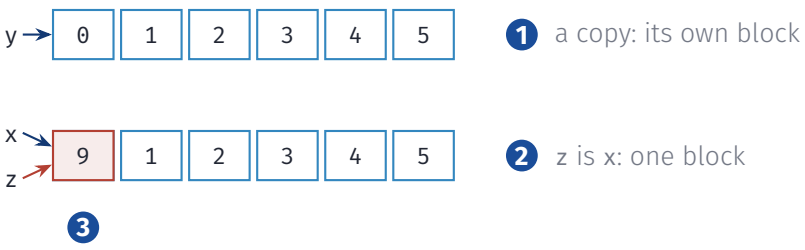

## A real copy

*Page 26.* Week 5 gave us several ways to copy a list. numpy's is simpler: you ask.

```
>>> x = np.arange(6)
>>> y = np.array(x)
>>> z = np.array(x, copy=False)
>>> x[0] = 9
>>> print(y)
[0 1 2 3 4 5]
>>> print(z)
[9 1 2 3 4 5]
```

`np.array(an_existing_array)` makes a copy by default: the safe default, and the opposite of `=`, which only adds a name. `copy=False` refuses to copy, and raises a `ValueError` when a copy cannot be avoided, as for a list.

1 `y = np.array(x)` copies the block, so `y` keeps `[0 1 2 3 4 5]`. 2 With `copy=False`, `z` is `x` itself. 3 `x[0] = 9` shows through `z`, never through `y`. Views are how numpy stays fast; when you want an array of your own, `np.array(...)` makes one.

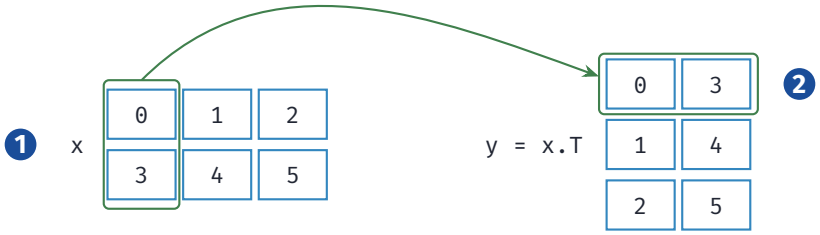


### Transpose: the free flip

*Page 27.* One more view, and a classic. `.T` reads the same block columns first:

```
>>> x = np.reshape(np.arange(6), (2, 3))
>>> y = x.T
>>> print(x)
[[0 1 2]
 [3 4 5]]
>>> print(y)
[[0 3]
 [1 4]
 [2 5]]
```

❶ `x` reads the block as two rows of three. ❷ `x.T` reads it columns first: `x`'s column 0, which holds 0 and 3, becomes `y`'s row 0, and every column becomes a row. Only the metadata changed, so the flip is instant whatever the size.




The café's week grid is days by hours; `grid.T` reads the same numbers as hours by days. Which is right? Neither: the question you ask picks the orientation.

**Common mistakes.** Reshaping to a size the data does not have: `np.arange(10).reshape(3, 3)` stops with `ValueError: cannot reshape array of size 10 into shape (3,3)`. Setting the shape directly, `x.shape = (2, 5)`: numpy 2.5 warns `Setting the shape on a NumPy array has been deprecated in NumPy 2.5.` Use `.reshape()` instead. And taking a reshape for a copy: a write through either name shows through both.

**Check yourself.**

- **3.1** A grid has shape `(5, 10)`. What are its `size` and its `len()`?
- **3.2** After these lines, does `a` change? Say which value, if any.

```
a = np.arange(6).reshape(2, 3)
b = a.T
b[0, 1] = 7
```

**See also.** Sheet 6, exercise 5. The Function reference, Week 6: Grids.

# 4 · Indexing and the week report

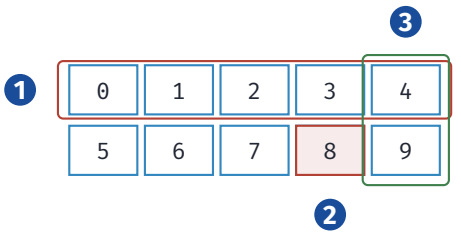
*Slides, pages 28 to 33.*

## Indexing in two dimensions

*Page 28.* A grid takes one index per dimension, row first, and every dimension takes week 4's slices:

```
>>> x = np.reshape(np.arange(10), (2, 5))
>>> print(x)
[[0 1 2 3 4]
 [5 6 7 8 9]]
>>> print(x[0])
[0 1 2 3 4]
>>> print(x[1, 3])
8
>>> print(x[:, -1])
[4 9]
```

❶ `x[0]` is row 0, the whole of it. ❷ `x[1, 3]` is one element, row 1 and column 3, which holds 8. It is really `x[(1, 3)]`: one tuple, one index per dimension, so you can index only the dimensions the shape has. ❸ `x[:, -1]` reads "every row, the last column": one slice, where a week-4 list of lists needs a loop to pull out one column.




`x[0]` is `[0 1 2 3 4]`

`x[1, 3]` is `8`

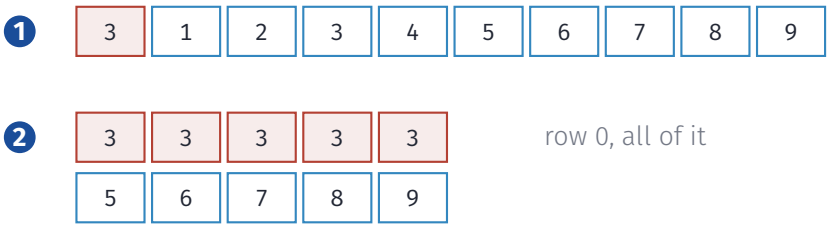
`x[:, -1]` is `[4 9]`

## Same syntax, a shape-dependent meaning

*Page 29.* Assignment indexes the same way, and the shape decides what you change:

```
>>> x = np.arange(10)
>>> x[0] = 3
>>> print(x)
[3 1 2 3 4 5 6 7 8 9]
>>> x = x.reshape(2, 5)
>>> x[0] = 3
>>> print(x)
[[3 3 3 3 3]
 [5 6 7 8 9]]
```

❶ On the flat array, `x[0] = 3` writes one seat. ❷ On the 2 × 5 grid, `x[0]` is a whole row, and the same line writes all five seats of it. Before you assign through an index, know your shape.




row 0, all of it

Slices assign in blocks. Here `np.repeat(0, 25)` builds the zeros, not `np.zeros(25)`, so the grid holds whole numbers and prints without dots:

```
>>> x = np.repeat(0, 25).reshape(5, 5)
>>> x[1:4, 1:4] = 1
>>> x[2, 2] = 2
>>> print(x)
[[0 0 0 0 0]
 [0 1 1 1 0]
 [0 1 2 1 0]
 [0 1 1 1 0]
 [0 0 0 0 0]]
```

`x[1:4, 1:4] = 1` sets the inner three-by-three in one statement, and `x[2, 2] = 2` sets the center.

## Slice drills on the week grid

*Page 30.* The café's week as a grid: five days as rows, Monday to Friday, and ten opening hours as columns, 08:00 to 17:00, items sold in each. The file, `data/hourly_items.csv`, starts like this:

```
h08,h09,h10,h11,h12,h13,h14,h15,h16,h17
34,28,17,12,26,31,14,9,11,8
38,31,15,14,29,27,12,11,9,7
...
```

numpy reads it in one line, explained with the week report below:

```
grid = np.genfromtxt("data/hourly_items.csv", skip_header=1, delimiter=",")
```

Write an expression for each of these before reading the answers:

1. All of Monday, the first row.
2. The 08:00 column, the first hour of every day.
3. Friday afternoon, the last row and the last five hours.
4. The Tuesday to Thursday lunch block, rows 1 to 3 and columns 4 and 5.
5. What does `grid[2, 3]` mean, in café English?

**Poll.** Write the slice for the 08:00 column, every day. It is `grid[:, 0]`: every row, column 0, which prints `[34. 38. 41. 36. 44.]`.

The answers: `grid[0]`; `grid[:, 0]`; `grid[4, 5:]`, or `grid[-1, -5:]`, since the last five hours, 13:00 to 17:00, are columns 5 to 9, and the 12:00 hour is column 4; `grid[1:4, 4:6]`, where each stop is one past the last row or column you want; and `grid[2, 3]` is the items sold on Wednesday in the 11:00 hour, since row 2 is the third day and column 3 the fourth hour. The fifth is the skill the exam tests: turning an index into its meaning.

## The week report

*Pages 31 to 33.* The owner's question: when do we need a second barista?

**Poll.** Facing the week grid, five days by ten hours: what would you ask it first? Many first questions fit. Its shape is a sound start, and the report prints it first; the owner's two, the busiest hour and the biggest day, are each an `argmax`.

`np.genfromtxt(file)` returns an array, and its options do the cleaning: `skip_header=1` drops the title row, and `delimiter=","` splits the columns. No `open()`, no `.split(",")`, no loop of `float()` calls: the file skills of weeks 2 to 4, in one line. Writing is the mirror, `np.savetxt(file, data)`.

Why does the grid print `34.`, with a dot, when the file holds 34? `genfromtxt` reads every number as a float unless told otherwise, whole numbers included. That is why the report prints its totals with `:.0f`.

A column is one hour across the whole week, so the plan is to sum each column and pick the biggest. Before the code below:

**Poll.** The hour totals are in `hour_totals`. Type the numpy call that gives the busiest hour's index. It is `hour_totals.argmax()`, the move from section 1.

```
def week_report():
    """Read the week's hourly-items grid and answer the owner's questions."""
    grid = np.genfromtxt(DATA_DIR + "hourly_items.csv", skip_header=1, delimiter=",")
    print(f"  Grid: {grid.shape[0]} days x {grid.shape[1]} hours")
    print(f"  Items sold this week: {grid.sum():.0f}")
    hour_totals = np.array([grid[:, hour].sum() for hour in range(grid.shape[1])])
    busiest = hour_totals.argmax()
    print(f"  Busiest hour: {OPENING_HOUR + busiest}:00 ({hour_totals[busiest]:.0f} items)")
    day_totals = np.array([grid[day].sum() for day in range(len(grid))])
    print(f"  Biggest day:  {DAYS[day_totals.argmax()]} ({day_totals.max():.0f} items)")
```

`grid.sum()` adds every cell. For each hour, `grid[:, hour]` is that hour down all five days, and its sum goes into `hour_totals`; `OPENING_HOUR + busiest` turns index 0 into 8:00. The same move by row gives the biggest day, and `DAYS`, the five weekday names at the top of the café file, turns its index into a name. Two roads lead to a biggest value: `hour_totals[busiest]` reads it at the index `argmax` found, and `day_totals.max()` asks for it directly. A comprehension and a column slice build the totals, with no `axis=`: section 6 says why.

Try it: start `python3 cafe_w6a.py` from your café folder, type `Ada` as the cashier, then choose `7`:

```
  Grid: 5 days x 10 hours
  Items sold this week: 1022
  Busiest hour: 8:00 (193 items)
  Biggest day:  Friday (243 items)
```

The totals behind those lines, in the shell, started from your café folder:

```
>>> import numpy as np
>>> grid = np.genfromtxt("data/hourly_items.csv", skip_header=1, delimiter=",")
>>> hour_totals = np.array([grid[:, hour].sum() for hour in range(grid.shape[1])])
>>> print(hour_totals)
[193. 163.  86.  66. 137. 156.  75.  51.  54.  41.]
>>> day_totals = np.array([grid[day].sum() for day in range(len(grid))])
>>> print(day_totals)
[190. 193. 206. 190. 243.]
```

❶ Each column's sum is one hour's total for the week. ❷ `argmax` picks the biggest, 193, at index 0: the 8:00 hour beats every other hour, so the second barista works mornings. ❸ Each row's sum is one day's total, and `argmax` picks Friday's 243, at index 4, the biggest day.



**In the project.** The dataset guide loads each market file with pandas, Friday's tool, then turns its column of closing prices into a numpy array, the kind these notes work with. The brief's plan for week 7 computes daily returns and a moving average from there, with plain numpy and loops, and the time-series supplement holds the recipes.

**Common mistakes.** Forgetting `skip_header=1`: the header line becomes a first row of `nan`, `[nan nan nan nan nan nan nan nan nan nan]`, and every column sum turns into `nan`. Forgetting `delimiter=","`: numpy splits on spaces, finds none, and reads each line as one value that is not a number, so the grid comes back as five `nan` with shape `(5,)`. Taking `len(grid)` for the number of cells: it is the number of rows, 5. And indexing a flat array with two numbers: `np.arange(10)[1, 3]` stops with `IndexError: too many indices for array: array is 1-dimensional, but 2 were indexed`.

**Check yourself.**

- **4.1** On the week grid, write the expression for Thursday's 12:00 hour, then for Monday's last three hours.
- **4.2** After these two lines, how many cells hold 5? And how many if the second line were `x[:, 0] = 5` instead?

```
x = np.repeat(0, 12).reshape(4, 3)
x[1] = 5
```

**See also.** Sheet 6, exercise 4. The Function reference, Week 6: Grids and Building arrays.

## 5 · Array multiplication

*Slides, pages 34 to 35.*

### Three ways to multiply

*Page 34.* numpy multiplies arrays in three ways, two of which you have already met:

```
a = np.repeat(2, 6).reshape(3, 2)
a * 3                            # 1. by a scalar: every element times 3
a * np.arange(6).reshape(3, 2)   # 2. by a same-shape array: element by element
a @ np.arange(6).reshape(2, 3)   # 3. by a compatible array: a dot product in every cell
```

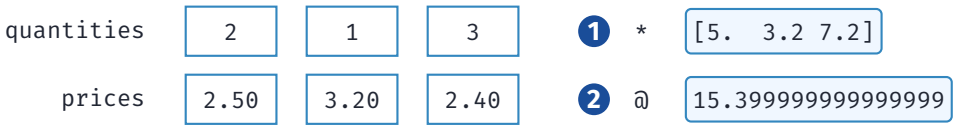
Multiplying by a scalar multiplies every element by that value. Two arrays of the same shape multiply element by element, the trace from section 2. With `@`, a $3 \times 2$ array and a $2 \times 3$ array give a $3 \times 3$ result, and every cell of it is a dot product: multiply pairs, then add them up, in one operation.

## The week-4 promise, paid

*Page 35.* Here is section 2's order again, the bill week 4 computed with a loop:

```
>>> quantities = np.array([2, 1, 3])
>>> prices = np.array([2.50, 3.20, 2.40])
>>> print(quantities * prices)
[5.  3.2 7.2]
>>> print((quantities * prices).sum())
15.399999999999999
>>> print(quantities @ prices)
15.399999999999999
```

Multiply each pair, add them up: that is what a bill is. ❶ `*` multiplies seat by seat and keeps the three line totals, an array. ❷ `@` multiplies seat by seat and adds the products up in the same pre-compiled pass: one number, the bill, in one line and one operator.




The float tail of section 2 comes along, and so do its fixes. For the receipt, format it: `f"{quantities @ prices:.2f}"` gives `15.40`. To compare it, use week 5's tolerance, never `==`:

```
>>> print(quantities @ prices == 15.4)
False
>>> print(abs(quantities @ prices - 15.4) < 1e-9)
True
```

Every backtest, every portfolio valuation and every weighted score you will compute is this same operator in a business suit.

**Common mistakes.** Using `*` where you meant `@`: on two flat arrays, `*` gives the line totals and `@` the bill. Mismatched shapes: `np.ones((3, 2)) @ np.ones((3, 2))` stops with a `ValueError` that ends `(size 3 is different from 2)`, because `@` needs the first array's columns to match the second array's rows.

**Check yourself.**

- **5.1** What shape does `np.ones((2, 3)) @ np.ones((3, 4))` have? And what happens with `np.ones((2, 3)) * np.ones((3, 4))`?
- **5.2** With `m = np.ones((2, 2))`, what are `m * m` and `m @ m`? Write each as numpy prints it.

**See also.** Sheet 6, exercise 2. The Function reference, Week 6: Arithmetic on the whole array.

# 6 · Walls and speed

*Slides, pages 36 to 37.*

## The walls of week 6

*Page 36.* Two walls to know. The first is the ragged grid from section 3: ragged rows raise the inhomogeneous-shape `ValueError`, and `genfromtxt` stops on a file with a short row, naming its line. A file whose third line holds two numbers where the others hold three gives:

```
ValueError: Some errors were detected !
    Line #3 (got 2 columns instead of 3)
```

numpy trades flexibility for speed, so the grid has to be a grid. The second wall is code from the internet. Search for "numpy sum columns" and you meet answers like these:

```
grid.sum(axis=0)          # per-column sums: Friday's tool
np.where(grid > 30)       # which positions match: Friday's tool too
```

They are numpy's own, and they arrive on Friday. Everything they do here, you already do with what you own: a column slice inside a comprehension.

```
np.array([grid[:, j].sum() for j in range(grid.shape[1])])   # per-column sums
```

The sheet's numpy half needs no `axis=`: column slices and reductions answer every exercise.

> **In the project.** The library policy draws a line of the same kind for the code you hand in: its method-level rule names the shortcuts that were not taught and stay out of a submission. Read it before you paste a line you found online.

## The timing race

*Page 37.* To use numpy well, stop thinking in elements and treat all the data as a single object. The payoff is measurable.

**Poll.** The register adds up a hundred million numbers twice, the Python way and the numpy way. How many times faster is numpy: 2×, 10×, 100×, or more? Make your guess before reading on.

The race, a sum of squares on a hundred million numbers:

```
size = 100000000
total = sum(x ** 2 for x in range(size))          # element thinking: about 4 s
total = np.sum(np.arange(size) ** 2)              # object thinking: about 0.07 s
```

The first line feeds Python's `sum()` a generator, week 8's tool, one number at a time. The second stays inside numpy from end to end. The slide's timings, about 4 seconds against about 0.07 on the same machine, make numpy about 60 times faster: of the poll's four options, 100× is the nearest. Your seconds and your ratio will differ. Do not run a hundred million on a laptop in a hurry: ten million shows the gap as clearly. And starting from a list, numpy loses at any size: building the array costs more than one sum saves.

Print the two totals and the race teaches a second lesson. Python's total comes first, exact, then numpy's:

```
333333328333333350000000
662921401752298880
```

numpy's integers have a fixed width, and a total past `2**63` wraps around to a wrong number without a word. Python's integers have no such limit, as week 1 promised, and that is one of the

things numpy trades for speed. Week 9 teaches how to predict costs like these before running anything.

**Common mistakes.** Pasting `axis=` code from the internet into the numpy half of the sheet: it works, and it skips the column-slice skill the sheet practices.

**Check yourself.**

- **6.1** Why does the sheet's numpy half need no `axis=`?
- **6.2** Started from a Python list of the same numbers, numpy loses the race for a single total, at any size. Why?

**See also.** Sheet 6, exercises 4 and 9. The Function reference, Week 6: Reductions.

## Where the café is now

*Slides, pages 38 to 41.*

The Daily Grind at the end of day 6: an unkillable register, `[6]` day stats from the live orders, and `[7]` a week report that schedules the second barista, all in a few lines of numpy. The finished file is `cafe_w6a.py`.

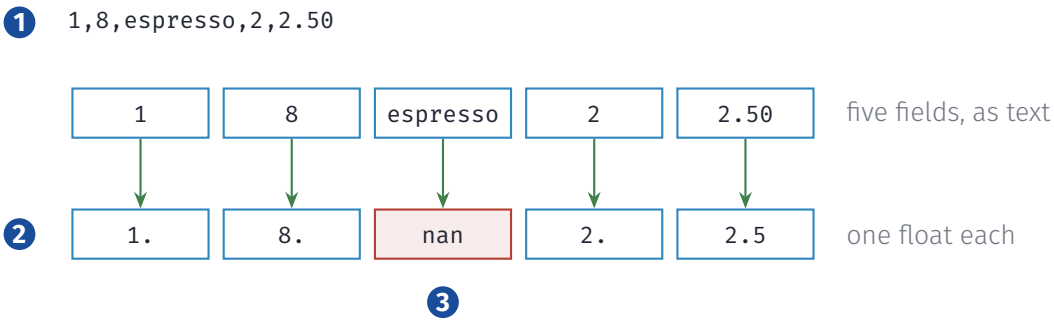
Our real history is `transactions.csv`, one sale per line:

```
day,hour,product,qty,price
1,8,espresso,2,2.50
1,8,croissant,1,2.80
...
```

**Poll.** One word: what will numpy do to our product names when it meets them? It turns each one into `nan`, "not a number":

```
>>> x = np.genfromtxt("data/transactions.csv", skip_header=1, delimiter=",")
>>> print(x[:3])
[[1.  8.  nan 2.  2.5]
 [1.  8.  nan 1.  2.8]
 [1.  9.  nan 1.  3.2]]
```

(1) `genfromtxt` splits each line at its commas into five fields, each still text. (2) It reads every field as a float, as it read the week grid. (3) `espresso` cannot be read as a number, so its seat gets `nan`, a float that marks the hole: numpy shrugged where the products were.



`nan` passes through arithmetic without a word: `x * x` squares every number and leaves each `nan` a `nan`.

```
>>> print((x * x)[:3])
[[ 1.   64.     nan  4.    6.25]
 [ 1.   64.     nan  1.    7.84]
 [ 1.   81.     nan  1.   10.24]]
```

`np.array` met text in section 1 by turning every number into text; `genfromtxt` meets it by turning the text into `nan`. Either way, the questions that matter most, the best seller and the revenue per product, live in exactly the column numpy cannot compute with. Friday brings pandas: tables that keep their labels and their words, read the whole history, group it and draw the charts.

**What the exam can ask from these slides**, as page 39 lists it: predict outputs, the reductions (`argmax` against `max`), reshape prints and slice results; trace the view trap (modify `one_d`, then say what `two_d` prints); write one-liners, such as "the total of column 2", "every price up 10%" or "the dot product of…"; tell `*` from `@` on the same two arrays; and spot the error in a ragged `np.array([[...], [...]])`.

**Go through the checklist on page 41.** Anything you cannot tick, read its section here again, then do the matching exercise in Part A of sheet 6, which the Monday practical of week 7 works. Self-check 6 opens on Friday evening and covers both of this week's lectures: its questions are the kinds the exam asks, so take it while the week is fresh.

## Practice it closed-book

The exam is closed book, so practice the way it runs: from memory, on paper, then check.

- Cover the outputs in sections 1, 2 and 3, and the pictures beside them, and write each one exactly as numpy prints it, spaces and dots included; then look.
- Redo page 25's trace from memory, with the picture covered: write its six lines, predict both prints, draw the one block and its two names, then run it to check. Then do it again on `np.arange(8)` reshaped to `(4, 2)`, writing `0` through the grid at `[2][1]`: predict which flat position changes, then run it.
- Write `week_report` from memory, drop it into a copy of the café, and run option 7.
- Explain in one sentence each why numpy is fast, what a view is, and how `*` differs from `@`. Check against sections 1, 3 and 5.
- In a few days, answer the check-yourself questions again without these notes, then take self-check 6 by hand, with Python closed.

## Answers to check yourself

- **1.1** It would show `np.float64(13.9)`: numpy's sum is a numpy number, and a dictionary prints its type. `float(...)` turns it into a plain Python float, which prints as `13.9`.
- **1.2** `x.mean()`. Both give `4999999.5`, but `x.mean()` runs pre-compiled over the one block of memory, while `sum(x)` makes Python fetch and check ten million numbers.
- **1.3** `[3.  2.5]`, then `<class 'numpy.float64'>`: the one float makes every element a float, so the 3 prints with its dot, and each element is a 64-bit numpy float.
- **2.1** `[10  7  4  1]` and `[1.  1.5 2. ]`. `arange` steps down by 3 from 10 and stops before 0; `linspace` puts three floats from 1 to 2, both ends included.
- **2.2** `menu * 0.8`, which prints `[1.92 2.4 ]`.
- **3.1** `size` is 50, the number of cells, and `len()` is 5, the number of rows.

- **3.2** Yes. `.T` is a view of the same block, so `b[0, 1]` is `a[1, 0]`, and `print(a)` shows the 7 there:

```
[[0 1 2]
 [7 4 5]]
```

- **4.1** `grid[3, 4]`, which holds `27.0`, then `grid[0, 7:]`, or `grid[0, -3:]`, which print as:

```
[ 9. 11.  8.]
```

- **4.2** Three: `x` has four rows of three, and `x[1] = 5` fills the whole of row 1. With `x[:, 0] = 5` instead, four: column 0 of each of the four rows.
- **5.1** `(2, 4)`. The `*` stops with `ValueError: operands could not be broadcast together with shapes (2,3) (3,4)`: element by element needs the same shape.
- **5.2** `m * m` multiplies cell by cell and keeps every 1. Each cell of `m @ m` adds the products of a row and a column, $1 \times 1 + 1 \times 1$, so every cell is 2:

```
[[1. 1.]
 [1. 1.]]
[[2. 2.]
 [2. 2.]]
```

- **6.1** A column slice inside a comprehension, the line under page 36's walls, gives the same per-column sums with tools taught already.
- **6.2** `np.array(the_list)` has to visit and check every element of the list, one object at a time, which is the very toll `sum()` pays, and only then does numpy sum. Measured from ten numbers to ten million, the conversion alone takes 7 to 10 times as long as `sum()` on the list. numpy pays off when the data is already an array, or when one array answers many questions.